Software Engineering Lab

Lab 25

Rate Limiting & Backpressure

Mengontrol Laju Masuk dan Mempropagasi Tekanan Beban di Sistem Terdistribusi



Lukman / lukman-ss

Antrean Tak Berbatas Memicu Kolaps Memori dan Latensi

Ketika laju kedatangan melebihi kapasitas proses hilir, antrean bertumpuk tanpa batas.

DAMPAK KETIADAAN LIMITER DAN BACKPRESSURE

Penumpukan Antrean

5.000.000 request dengan drain 2.000/detik butuh 41 menit 40 detik untuk menguras buffer.

DAMPAK KETIADAAN LIMITER DAN BACKPRESSURE

Kehabisan Memori

Buffer tak terbatas menyerap jutaan objek hingga proses terbunuh oleh OOM killer OS.

DAMPAK KETIADAAN LIMITER DAN BACKPRESSURE

Bukan Berarti Rusak

Sistem melayani request normal saat sepi, tetapi bukan berarti aman dari lonjakan mendadak.

Ketiadaan batas masuk membiarkan jutaan request menumpuk saat downstream melambat. Sistem yang berjalan lancar di kondisi normal bukan berarti mampu bertahan tanpa rate limiter ketika lonjakan beban traffic tak terduga datang.

Unbounded buffers trade bounded latency for catastrophic out-of-memory crashes.

Throttling Berbasis IP dan Retry Deterministik Melumpuhkan Sistem

CGNAT mencampur ribuan tenant, sementara retry tanpa jitter memicu thundering herd.

KELEMAHAN PENDEKATAN NAIF

Kolateral CGNAT

RFC 6598 membuat ribuan pelanggan berbagi satu IP; satu pengguna bising melumpuhkan seluruh tenant.

KELEMAHAN PENDEKATAN NAIF

Thundering Herd

100 client retry di interval deterministik yang sama menggandakan beban saat backend baru pulih.

KELEMAHAN PENDEKATAN NAIF

Tidak Otomatis Aman

Menambah kapasitas server tidak otomatis menyelesaikan bottleneck jika retry storm terus menggempur.

Rate limiting berbasis IP publik merugikan ribuan pengguna di balik CGNAT karena kuota terpakai habis oleh satu pengguna. Di sisi lain, menambah kapasitas CPU tidak otomatis menyelesaikan masalah jika retry klien terus datang serempak.

IP-based rate limits cause collateral damage under CGNAT; jitterless retries synchronize thundering herds.

Tiga Pilar Resiliensi: Limiter, Bounded Queue, dan Jitter

Pengendalian pintu masuk, penolakan cepat internal, dan desinkronisasi percobaan ulang.

BOUNDARY GUARD

1. Rate Limiting

Token bucket mengontrol laju masuk di API gateway dan memotong burst liar.

INTERNAL DEFENSE

2. Backpressure

Bounded queue menolak kelebihan job seketika dengan sinyal ErrQueueFull.

CLIENT BACKOFF

3. Jittered Retry

Hanya pada jalur ini acakan [0, temp] mencegah sinkronisasi thundering herd.

PROTOCOL SIGNAL

4. RFC 6585 429

Memberi tahu klien kapan waktu aman untuk mencoba kembali via Retry-After.

Resiliensi sistem terdistribusi bertumpu pada isolasi pintu masuk via token bucket dan propagasi tekanan ke hulu via bounded queue. Hanya pada jalur ini klien yang menerima sinyal 429 mendistribusikan waktu tunggu secara acak merata.

Rate limiting polices external boundaries; backpressure rejects internal overload; jitter disperses client retries.

Alur Arsitektur: Dari Ekstraksi Tenant ke Konsumer

Memisahkan layer gateway, rate limiter, bounded buffer, hingga worker pool.

HTTP MIDDLEWARE

1. Tenant Extractor

Membaca header X-API-Key untuk mendapatkan identitas tenant terisolasi.

REGISTRY LOOKUP

2. Token Bucket

Memeriksa kuota token; melebihi batas menghasilkan 429 Too Many Requests.

CHANNEL BUFFER

3. Bounded Queue

Menerima job ke antrean; buffer penuh langsung menolak dengan ErrQueueFull.

JOB CONSUMER

4. Worker Pool

Konsumer memproses job; belum membuktikan isolasi resource CPU per worker.

Request masuk diidentifikasi per tenant, divalidasi oleh token bucket, lalu diserahkan ke bounded queue. Arsitektur ini mencegah kelebihan beban di memori, namun belum membuktikan jaminan alokasi CPU per tenant di level container OS.

Layered defense isolates tenants at the gateway and sheds excess load immediately at queue boundaries.

Perbandingan Algoritma: Token Bucket vs Leaky Bucket

Keduanya setara secara matematis sebagai meter, namun berbeda sudut pandang kendali.

Karakteristik	Token Bucket (Traffic Burst)	Leaky Bucket (Traffic Smoothing)
Toleransi Burst	Mengizinkan burst hingga kapasitas B	Menahan burst; output rata pada laju R
State Utama	Jumlah token yang siap dikonsumsi	Level air yang menunggu dibocorkan
Mekanisme Tolak	Token < n yang diminta	Penambahan air melebihi kapasitas B
Kasus Penggunaan	API Gateway, bursty interactive user	Egress traffic shaping, NGINX limit_req
Respons Standar	HTTP 429 Too Many Requests	HTTP 429 atau 503 Service Unavailable

Token bucket mengizinkan akumulasi token untuk mengakomodasi burst jangka pendek, sedangkan leaky bucket memaksa laju keluaran konstan. Pilihan bergantung pada kebutuhan: fleksibilitas burst atau penghalusan trafik ke hilir.

Token bucket permits bursts up to capacity; leaky bucket smooths output to a strict constant rate.

Token Bucket: Pengisian Waktu Berjalan dan Konsumsi

Menghitung elapsed time secara monotonik tanpa timer background yang boros.

```
func (tb *TokenBucket) AllowN(n float64) bool {
    tb.mu.Lock()
    defer tb.mu.Unlock()
    now := time.Now()
    elapsed := now.Sub(tb.lastRefill).Seconds()
    tb.tokens = math.Min(tb.capacity, tb.tokens+elapsed*tb.refillRate)
    tb.lastRefill = now
    if tb.tokens >= n {
        tb.tokens -= n
        return true
    }
    return false
}
```

TokenBucket menghitung penambahan token secara pasif berdasarkan selisih waktu antar-panggilan. Penggunaan mutex bukan berarti implementasi tidak optimal; lock lokal menjamin atomisitas pembaruan multi-field secara bersih dan aman.

Lazy replenishment on call converts expensive timer ticker goroutines into cheap timestamp arithmetic.

internal/ratelimit/bucket.go:29–46

Kalkulasi Retry-After: Panduan Presisi Waktu Tunggu

Menghitung waktu tunggu pembulatan ke atas agar klien tidak retry prematur.

```
func (tb *TokenBucket) RetryAfterSeconds(n float64) int {
    tb.mu.Lock()
    defer tb.mu.Unlock()
    // ... perbarui token dari elapsed time ...
    if tb.tokens >= n { return 0 }
    needed := n - tb.tokens
    secs := needed / tb.refillRate
    rounded := int(secs)
    if float64(rounded) < secs { rounded++ }
    return rounded
}
```

Kekurangan token dibagi dengan laju pengisian lalu dibulatkan ke atas menjadi detik bulat. Hanya pada jalur ini klien yang ditolak menerima informasi presisi kapan kuota mereka akan terisi kembali tanpa menebak-nebak.

Round up retry delay to integer seconds to prevent clients from retrying before token replenishment.

internal/ratelimit/bucket.go:55–79

Bounded Queue: Penolakan Cepat dengan Select-Default

Mencegah tumpukan memori tak terbatas dengan membuang beban berlebih seketika.

```
func (bq *BoundedQueue) TrySubmit(job Job) error {
    bq.stopMu.RLock()
    defer bq.stopMu.RUnlock()
    if bq.stopped { return ErrQueueStopped }
    select {
    case bq.queue <- job:
        bq.accepted.Add(1)
        return nil
    default:
        bq.rejected.Add(1)
        return ErrQueueFull
    }
}
```

Pola select-default mengantrekan job jika kanal longgar atau langsung mengembalikan ErrQueueFull saat penuh. Menolak job seketika bukan berarti sistem tidak andal; ini tindakan proteksi mandiri agar proses tidak kehabisan memori.

Select-default on bounded channels gives zero-allocation, instant backpressure shedding.

AWS Full Jitter: Mencegah Sinkronisasi Thundering Herd

sleep = random(0, min(cap, base × 2^attempt)) menyebarkan lonjakan beban.

```
func ComputeBackoff(strategy BackoffStrategy, attempt int,
    cfg Config, prevSleep time.Duration) time.Duration {
    expBackoff := float64(cfg.Base) * math.Pow(2, float64(attempt))
    temp := math.Min(float64(cfg.Cap), expBackoff)
    switch strategy {
    case FullJitter:
        if temp <= 0 { return 0 }
        return time.Duration(rand.Float64() * temp)
    // ... case EqualJitter, DecorrelatedJitter, NoJitter ...
    }
}
```

Full Jitter mengambil sampel acak seragam antara nol hingga batas plafon eksponensial. Menambahkan jitter tidak otomatis mempercepat waktu pemulihan jika kapasitas downstream memang telah runtuh secara permanen.

Full jitter samples uniformly from zero to the exponential ceiling to disperse client retry waves.

Middleware HTTP: Menegakkan Status 429 RFC 6585

Mengekstraksi tenant API key dan mengembalikan header Retry-After standar.

```
func RateLimitMiddleware(reg *ratelimit.Registry, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        key := r.Header.Get("X-API-Key")
        if key == "" { key = "anonymous" }
        bucket := reg.Get(key)
        if !bucket.Allow() {
            retryAfter := bucket.RetryAfterSeconds(1.0)
            w.Header().Set("Retry-After", strconv.Itoa(retryAfter))
            w.WriteHeader(http.StatusTooManyRequests) // 429
            _ = json.NewEncoder(w).Encode(RateLimitResponse{...})
            return
        }
        next.ServeHTTP(w, r)
    })
}
```

Middleware mengisolasi token bucket per tenant key sebelum meneruskan request ke handler berikutnya. Mengirimkan status 429 bukan berarti client dilarang selamanya; header Retry-After memandu kapan pemanggilan boleh diulang.

Extract tenant keys at edge middleware, reject excess with 429, and provide explicit Retry-After headers.

internal/httputil/middleware.go:17–39

Eksekusi Demo cmd/demo: Empat Verifikasi Perilaku

Output terminal membuktikan burst, smoothing, penolakan beban, dan sebaran jitter.

```
=== 1. Token Bucket Burst & Rate Limiting ===
Request #1..3: Allowed=true | Request #4..5: Allowed=false
After 300ms pause: Allowed=true (Remaining Tokens: 0.5)
=== 2. Leaky Bucket Traffic Smoothing ===
Request #1..3: Allowed=true | Request #4..5: Allowed=false (Water: 3.0)
=== 3. Bounded Queue Backpressure (Load Shedding) ===
Job #1..3: ACCEPTED | Job #4, #6: REJECTED (queue capacity exceeded)
Stats: Accepted=4, Rejected=2, Processed=1
=== 4. AWS Retry Backoff Strategies (Attempts 0..3) ===
Attempt 0 -> NoJitter: 100ms  | FullJitter: 61ms   | EqualJitter: 80ms
Attempt 3 -> NoJitter: 800ms  | FullJitter: 342ms  | EqualJitter: 647ms
```

Demo membuktikan seluruh skenario berjalan sesuai spesifikasi: 3 request awal lolos burst lalu ditolak, buffer antrean menolak job saat melampaui kapasitas 3, dan Full Jitter mendispersi waktu jeda secara acak.

Executable verification proves burst accommodation, queue shedding, and jitter dispersion in live runs.

cmd/demo/main.go:13–65

Verifikasi Test Suite: 15/15 PASS dan Bersih dari Race

Seluruh modul rate limiting, queue, retry, dan middleware teruji konkurensi.

```
=== RUN   TestTokenBucket_BurstAndRefill / ConcurrencyRace
--- PASS: TestTokenBucket_* (0.05s)
=== RUN   TestLeakyBucket_LeakRate / ConcurrencyRace
--- PASS: TestLeakyBucket_* (0.02s)
=== RUN   TestRegistry_TenantIsolation / ConcurrentSameKeyGet
--- PASS: TestRegistry_* (0.01s)
=== RUN   TestBoundedQueue_RejectionUnderLoad / ConcurrencySafety
--- PASS: TestBoundedQueue_* (0.08s)
=== RUN   TestComputeBackoff_Bounds / DecorrelatedJitter_Bounds
--- PASS: TestComputeBackoff_* (0.00s)
=== RUN   TestRateLimitMiddleware_RFC6585 / AnonymousFallback
--- PASS: TestRateLimitMiddleware_* (0.00s)
PASS: ok labs/25-rate-limiting-and-backpressure (race detector: clean)
```

Seluruh 15 unit test lolos tanpa kegagalan di bawah race detector Go dengan goroutine agresif. Simulasi in-memory ini tidak sama dengan cluster Redis terdistribusi yang harus menangani latency jaringan antar node.

Race-clean unit tests prove concurrent synchronization correctness in local shared-memory runtimes.

engineering/03-execution-result.md:8-60

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Transparansi batasan rekayasa antara simulator lab lokal dan sistem produksi.

BATAS PEMBUKTIAN REKAYASA

YANG DIBUKTIKAN

Penolakan burst token bucket, fast drop bounded queue, dan sebaran rentang Full Jitter.

BATAS PEMBUKTIAN REKAYASA

YANG TIDAK DIBUKTIKAN

Sinkronisasi token lintas multi-instance pod tanpa shared state eksternal.

BATAS PEMBUKTIAN REKAYASA

BATAS KLAIM EMPIRIS

Lab belum membuktikan ketahanan cluster terhadap badai DDoS ratusan ribu node eksternal.

Lab ini membuktikan kebenaran matematis algoritma bucket, isolasi tenant lokal, dan pemotongan beban antrean. Namun, pengujian lokal ini belum membuktikan ketahanan cluster multi-region terhadap serangan DDoS skala masif.

Clearly delimit verified algorithmic invariants from unproven distributed scale heuristics.

Trade-off Arsitektur Rate Limiting & Backpressure

Kompromi antara kepuasan pengguna, akurasi sinkronisasi, dan kompleksitas sistem.

KOMPROMI REKAYASA SISTEM

False Rejections

Limiter agresif berisiko menolak pengguna legitimate saat aktivitas bisnis melonjak wajar.

KOMPROMI REKAYASA SISTEM

Shared State Latency

Sinkronisasi Redis menambah latensi 1–2ms di hot-path setiap request HTTP yang masuk.

KOMPROMI REKAYASA SISTEM

Tidak Otomatis Kebal

Memasang middleware limiter tidak otomatis membuat database kebal terhadap query N+1 berat.

Rate limiting mencegah kehabisan kapasitas tetapi berisiko menolak transaksi sah pengguna di saat peak season. Menambahkan limiter di pintu gerbang tidak otomatis melindungi storage jika kueri internal tidak terindeks dengan baik.

Rate limiting trades edge false-rejection risks for core service survival and latency stability.

Analisis Beban: Little's Law dan Pertumbuhan Backlog

L = λ × W membuktikan bagaimana penumpukan antrean merusak latensi sistem.

FORMULA MATEMATIKA SISTEM ANTREAN

Little's Law (L = λW)

Kapasitas in-flight (L) adalah perkalian antara laju kedatangan (λ) dan durasi proses (W).

FORMULA MATEMATIKA SISTEM ANTREAN

Akumulasi Backlog

Pertumbuhan antrean = (Arrival Rate – Processing Rate) × Waktu durasi kelebihan beban.

FORMULA MATEMATIKA SISTEM ANTREAN

Kalkulasi Lab 25

5.000.000 request ÷ 2.000 drain/detik = 2.500 detik (41 menit 40 detik); hanya pada jalur bounded buffer batas latensi terjaga.

Jika laju kedatangan melampaui laju pemrosesan, waktu tunggu membengkak hingga puluhan menit. Hanya pada jalur dengan bounded queue penumpukan dipotong seketika sehingga latensi job yang diterima tetap dalam batas SLA.

L = lambda * W: when processing capacity is saturated, queue wait times degrade proportionally to backlog depth.

Studi Kasus Stripe: Arsitektur Limiter 4 Lapisan

Pola pembagian tanggung jawab proteksi API gateway pada skala pembayaran global.

PER-USER QUOTA

1. Request Rate Limiter

Membatasi laju request per pengguna menggunakan token bucket terdistribusi.

IN-FLIGHT BOUND

2. Concurrent Limiter

Membatasi jumlah request simultan yang sedang aktif berjalan per pengguna.

CRITICALITY RESERVE

3. Fleet Load Shedder

Menolak trafik berprioritas rendah demi menjamin kapasitas transaksi kritis.

EXECUTION GUARD

4. Worker Load Shedder

Membuang uji coba test-mode, GET lalu POST; bukan berarti request hilang tanpa status.

Stripe menerapkan 4 tier limiter independen untuk mendegradasi sistem secara anggun saat kelebihan beban. Membuang trafik berprioritas rendah saat CPU tinggi bukan berarti sistem gagal, melainkan langkah penyelamatan transaksi keuangan kritis.

Layer multiple specialized limiters to drop low-criticality traffic first during severe overload.

Google SRE: Criticality Level dan Kuota Retry

Mencegah retry storm cascading failure melalui pembatasan rasio percobaan ulang.

DISIPLIN KETAHANAN GOOGLE SRE

4 Level Criticality

CRITICAL_PLUS, CRITICAL, SHEDDABLE_PLUS, SHEDDABLE; tolak level rendah lebih dulu.

DISIPLIN KETAHANAN GOOGLE SRE

Retry Budget Klien

Maksimal 3 attempt per-request dan rasio retry maksimal 10% dari total trafik klien.

DISIPLIN KETAHANAN GOOGLE SRE

Token Bucket Quota

AWS SDK mengadopsi 500 token retry pool; cakupannya mengikuti test kalibrasi error transien.

Google SRE membagi request ke 4 tingkat kepentingan dan membatasi retry maksimal 10% dari volume panggilan. Strategi kuota retry token bucket ini diadopsi luas di industri, dan cakupannya mengikuti test evaluasi latensi backend.

Enforce strict retry budgets (max 3 attempts, <10% retry ratio) to eliminate cascading retry storms.

Transformasi Desain: Unprotected vs Resilient Pipeline

Pergeseran dari sistem rentan crash ke arsitektur dengan degradasi terukur.

SEBELUM (UNPROTECTED) VS SESUDAH (RESILIENT)

Sebelum: Unbounded Channel

Antrean bertambah hingga jutaan item; latensi membengkak 40+ menit sebelum proses OOM crash.

SEBELUM (UNPROTECTED) VS SESUDAH (RESILIENT)

Sesudah: Bounded + Jitter

Kelebihan beban ditolak cepat (429/ErrQueueFull); klien desinkronisasi via Full Jitter.

SEBELUM (UNPROTECTED) VS SESUDAH (RESILIENT)

Bukan Berarti Nol Error

Resiliensi bukan berarti bebas error 100%, melainkan memastikan sistem tetap hidup melayani trafik sehat.

Sebelum proteksi, lonjakan trafik menelan seluruh memori dan mematikan server. Menerapkan limiter dan backpressure bukan berarti meniadakan error, melainkan mengendalikan error secara sadar agar layanan inti tetap beroperasi.

Transform uncontrolled systemic collapse into bounded, graceful rejection under peak overload.

Checklist Rekayasa Rate Limiting & Backpressure

Sepuluh kriteria wajib verifikasi sebelum merilis layanan ke traffic produksi.

- ✓ Identifikasi batas sistem: tentukan rate limiter di edge API gateway dan service mesh.
- ✓ Pilih algoritma yang tepat: Token Bucket untuk burst wajar, Leaky Bucket untuk output konstan.
- ✓ Gunakan kunci identitas per-tenant (X-API-Key atau User ID) untuk mencegah kolateral CGNAT.
- ✓ Terapkan Bounded Queue dengan kapasitas terukur sesuai laju pemrosesan konsumer downstream.
- ✓ Kembalikan respons standar HTTP 429 dengan header Retry-After presisi (RFC 6585).
- ✓ Gunakan Exponential Backoff dengan Full Jitter pada semua klien pemanggil untuk mencegah thundering herd.
- ✓ Tetapkan Retry Budget: maksimal 3 percobaan per request dan batas rasio retry di bawah 10%.
- ✓ Pantau Queue Age, bukan hanya Queue Length, untuk deteksi dini pembengkakan latensi pemrosesan.

Apply strict architectural gates across limits, buffers, retries, and metrics before production rollout.

Anti-Pattern Umum: Kesalahan Fatal Pengendalian Overload

Kekeliruan perancangan yang sering memicu pemadaman massal di industri.

HINDARI KESALAHAN ARSITEKTURAL INI

IP Limiting di CGNAT

Membatasi IP publik memblokir ribuan pengguna sah di balik satu gerbang ISP seluler.

HINDARI KESALAHAN ARSITEKTURAL INI

Retry Tanpa Jitter

Interval eksponensial kaku menyinkronkan ribuan klien menjadi gelombang thundering herd penghancur.

HINDARI KESALAHAN ARSITEKTURAL INI

Unbounded Buffer

Antrean tanpa batas bukan berarti aman; itu bom waktu memori yang menjamin kegagalan total.

Throttling berbasis IP merugikan tenant di balik CGNAT, sedangkan retry deterministik melipatgandakan beban downstream saat recovery. Memberi buffer antrean tak terbatas bukan berarti sistem toleran, melainkan menimbun kebangkrutan memori.

Avoid IP-only throttling, eliminate deterministic retries, and never allow unbounded queues.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Validasi penuh tanpa dependensi eksternal pihak ketiga (pure Go standard library).

```
cd labs/25-rate-limiting-and-backpressure
go test -v ./...    # validasi unit test bucket, queue, retry, dan middleware
go test -race ./... # verifikasi konkurensi stateless bebas data race
go run ./cmd/demo   # eksekusi demonstrasi langsung 4 pilar arsitektur
```

Seluruh mekanisme rate limiting dan backpressure ditulis murni menggunakan Go standard library tanpa framework pihak ketiga. Pengujian lokal ini memverifikasi algoritma, namun production masih perlu Redis untuk koordinasi multi-pod.

Zero-dependency local verification guarantees fast, repeatable validation across developer workflows.

README.md:55-70

Prinsip Inti: Kendalikan Pintu Masuk, Propagasikan Tekanan

Arsitektur bertahan lama ketika batas beban ditegakkan secara disiplin di setiap lapisan.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Polisikan Pintu Gerbang

Gunakan token bucket per tenant untuk membatasi kuota sebelum request membebani resource inti.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Tolak Cepat di Hilir

Terapkan bounded queue agar kelebihan beban dibuang seketika tanpa menumpuk latensi.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Desinkronisasi Klien

Terapkan Full Jitter; tidak otomatis menjamin backend pulih jika downstream rusak permanen.

Rate limiting menjaga gerbang masuk, sementara backpressure mempropagasikan sinyal tekanan balik ke hulu. Menerapkan Full Jitter pada klien tidak otomatis memulihkan sistem jika kapasitas downstream memang tidak memadai untuk beban dasar.

Police external ingress, reject internal overload fast, and desynchronize client retries with jitter.

Sistem tanpa backpressure adalah sistem yang menunda kebangkrutan memorinya sendiri.

A system without backpressure is merely borrowing time until an unavoidable memory collapse.

Software Engineering Lab 25 – Rate Limiting & Backpressure

internal/ratelimit/bucket.go • internal/ratelimit/registry.go
internal/backpressure/queue.go • internal/retry/backoff.go
internal/httputil/middleware.go • cmd/demo/main.go

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/25-rate-limiting-and-backpressure`

Lukman / lukman-ss

Commit: 45cc429 • Status: APPROVED
Test passing: 15 unit tests (burst, refill, tenant isolation, queue shedding, jitter)
Race detector: Clean (go test -race ./... PASS in 0.28s)
Implementasi Go standard library murni; in-memory single process architecture.